

ОТЧЕТ ПО HW4

Кажкаримов Асхат, БПИ-233, aakazhkarimov@edu.hse.ru

Я претендую на оценку до 10 баллов, генеративный ИИ не был использован для работы над этим заданием.

Дополнительно код работы можно найти по ссылке:

<https://github.com/justcipunz/mydrive>

Цель работы

Цель работы - реализовать многопользовательское клиент-серверное файловое хранилище MyDrive на TCP. Клиент должен синхронизировать локальную директорию с сервером. Сервер должен принимать подключения нескольких клиентов, сравнивать файлы по имени, размеру и контрольной сумме, принимать недостающие и измененные файлы и хранить их в отдельной директории пользователя.

Дополнительно в работе реализована параллельная передача файлов через несколько TCP-соединений и режим передачи файлов с использованием DMA.

Что использовалось

Работа выполнена на Java 17.

Сборка проекта выполняется через Maven.

Для сетевого взаимодействия использовался Netty.

Для сериализации служебных сообщений использовался Jackson.

Для анализа TCP-трафика использовался WireShark.

Структура программы

Основной код находится в пакете mydrive.

Основные пакеты

mydrive.client

Содержит клиентскую часть программы: запуск клиента, чтение конфигурации, консольные команды, сканирование директории и отправку файлов.

mydrive.server

Содержит серверную часть программы: запуск сервера, обработку TCP-соединений, сравнение файлов, прием файлов и сохранение данных в хранилище.

mydrive.common

Содержит общий код: модели сообщений, кодирование и декодирование протокола, расчет SHA-256 и запись метрик передачи.

Основные классы

ClientMain

Запускает клиентское приложение. При старте загружает client.properties, выводит параметры клиента и запускает консольный цикл команд.

ClientConfig

Загружает настройки клиента:

- client.id;
- client.id.file;
- client.dir;
- server.ip;
- server.port;
- max.connections;
- transfer.mode;

- `metrics.csv.path`.

Если `client.id` не указан, клиент создает постоянный идентификатор и сохраняет его в файл `client.id.file`.

ConsoleCommandLoop

Читает команды пользователя из консоли.

Реализованы команды:

- `sync` - выполнить синхронизацию;
- `show-config` - показать конфигурацию клиента;
- `show-metrics-path` - показать путь к файлу метрик;
- `exit` - завершить клиент.

DirectoryScanner

Сканирует клиентскую директорию. Поддиректории не обрабатываются. Для каждого файла собираются:

- имя файла;
- размер;
- SHA-256.

MyDriveClient

Управляет одним раундом синхронизации. Клиент подключается к серверу, отправляет ID, отправляет список файлов, получает список файлов для передачи и запускает отправку файлов.

SyncCoordinator

Выполняет управляющий обмен с сервером:

- отправляет HELLO;
- отправляет FILE_LIST;
- получает MISSING_FILES;
- отправляет SYNC_PLAN.

FileTransferClient

Отправляет один файл на сервер. В режиме NON_DMA файл передается частями. В режиме DMA файл передается через DefaultFileRegion.

ServerMain

Запускает серверное приложение.

ServerConfig

Загружает настройки сервера:

- server.bind.ip;
- server.bind.port;
- server.storage.root.

MyDriveServer

Создает TCP-сервер на Netty и принимает подключения клиентов.

ClientSessionHandler

Обрабатывает сообщения клиента на сервере. Принимает HELLO, FILE_LIST, SYNC_PLAN, FILE_TRANSFER_START, части файла и завершение передачи.

ServerDiffUtil

Сравнивает список файлов клиента со списком файлов сервера. Если файл отсутствует, изменился по размеру или изменился по SHA-256, он добавляется в список файлов для передачи.

ServerStorageService

Работает с серверным хранилищем. Для каждого клиента используется отдельная папка:

storage/{clientId}

ReconciliationService

Удаляет лишние файлы с сервера, которых больше нет у клиента. Также проверяет имена файлов, чтобы клиент не мог выйти за пределы своей серверной директории.

SyncSessionRegistry

Хранит состояние текущей синхронизации. После получения всех файлов завершает синхронизацию и отправляет клиенту SYNC_COMPLETE.

ProtocolEncoder и ProtocolDecoder

Реализуют собственный протокол поверх TCP. Для Java используется ByteToMessageDecoder, так как TCP может передавать данные частями.

ChecksumService

Считает SHA-256 для файлов.

MetricsCsvWriter

Записывает метрики передачи файлов в CSV-файл.

Команды программы

Клиент поддерживает команды:

- sync - синхронизировать директорию с сервером;
- show-config - вывести конфигурацию;
- show-metrics-path - вывести путь к CSV с метриками;
- exit - завершить работу клиента.

Конфигурация клиента

Пример client.properties:

client.id=

`client.id.file=./mydrive-client-id`

`client.dir=./client-data`

`server.ip=127.0.0.1`

`server.port=9000`

`max.connections=4`

`transfer.mode=NON_DMA`

`metrics.csv.path=./metrics/transfer-metrics.csv`

Параметр `max.connections` задает максимальное количество одновременных подключений к серверу. Поддерживаются значения от 1 до 32.

Параметр `transfer.mode` может принимать значения:

- `NON_DMA`;
- `DMA`.

Конфигурация сервера

Пример `server.properties`:

`server.bind.ip=0.0.0.0`

`server.bind.port=9000`

`server.storage.root=./storage`

Сервер слушает порт 9000. Файлы клиентов сохраняются в папке `storage`.

Описание протокола

Для передачи служебных сообщений реализован собственный протокол поверх TCP.

Формат кадра:

- `frameLength` - длина кадра;
- `messageType` - тип сообщения;
- `requestId` - идентификатор запроса;
- `payload` - сериализованные данные сообщения.

Служебные сообщения сериализуются в массив байт через Jackson. После этого `ProtocolEncoder` записывает байты в TCP-соединение.

Так как TCP не сохраняет границы сообщений, на стороне получателя используется `ProtocolDecoder`. Он сначала читает длину сообщения, затем ждет, пока придут все байты кадра. Только после этого сообщение передается дальше в обработчик.

Используются такие типы сообщений:

- HELLO;
- FILE_LIST;
- MISSING_FILES;
- SYNC_PLAN;
- FILE_TRANSFER_START;
- FILE_TRANSFER_CHUNK;
- FILE_TRANSFER_ACK;
- FILE_TRANSFER_COMPLETE;
- SYNC_COMPLETE;
- ERROR.

Ключевые фрагменты реализации протокола

Основная часть протокола реализована в `ProtocolEncoder` и `ProtocolDecoder`.

`ProtocolEncoder` формирует кадр сообщения. Сначала записывается длина кадра, затем тип сообщения, идентификатор запроса и `payload`:

@Override

```
protected void encode(ChannelHandlerContext ctx, ProtocolFrame frame, ByteBuf
out) {

    int frameLength = 1 + Long.BYTES + frame.payload().length;

    out.writeInt(frameLength);

    out.writeByte(frame.type().code());

    out.writeLong(frame.requestId());

    out.writeBytes(frame.payload());

}
```

Так клиент и сервер передают не объект Java, а набор байт поверх TCP.

ProtocolDecoder решает обратную задачу. Он сначала читает длину кадра. Если в TCP-буфере еще нет всех байт сообщения, decoder откатывает позицию чтения и ждет следующую часть данных:

```
in.markReaderIndex();

int frameLength = in.readInt();

if (in.readableBytes() < frameLength) {

    in.resetReaderIndex();

    return;

}
```

После получения полного кадра decoder читает тип сообщения, requestId и payload:

```
byte typeCode = in.readByte();
```



```
long requestId = in.readLong();
```

```
byte[] payload = new byte[frameLength - 1 - Long.BYTES];
```

```
in.readBytes(payload);
```

```
MessageType type = MessageType.fromCode(typeCode);
```

```
out.add(new ProtocolFrame(type, requestId, payload));
```

Перед отправкой служебные сообщения сериализуются через Jackson:

```
public static ProtocolFrame of(MessageType type, long requestId, Object payload)
{
    return new ProtocolFrame(type, requestId, ProtocolJson.write(payload));
}
```

В режиме NON_DMA содержимое файла передается частями через сообщения FILE_TRANSFER_CHUNK:

```
FileTransferChunkMessage chunkMessage =
    new FileTransferChunkMessage(transferId, chunk);
```

```
channel.writeAndFlush(
```

```
    ProtocolFrameUtil.of(MessageType.FILE_TRANSFER_CHUNK, requestId,
    chunkMessage)
).sync();
```

В режиме DMA после служебного сообщения FILE_TRANSFER_START файл передается через DefaultFileRegion:

```
try (RandomAccessFile raf = new RandomAccessFile(file.toFile(), "r");  
    FileChannel fc = raf.getChannel()) {  
    channel.writeAndFlush(new DefaultFileRegion(fc, 0, fc.size())).sync();  
}
```

На сервере для режима DMA после FILE_TRANSFER_START decoder убирается из pipeline, и сервер принимает raw bytes файла до заранее известного размера:

```
if (dmaMode && ctx.pipeline().get(ProtocolDecoder.class) != null) {  
    ctx.pipeline().remove(ProtocolDecoder.class);  
}
```

Граница файла определяется по полю sizeBytes из FILE_TRANSFER_START.

Режим NON_DMA

В режиме NON_DMA файл передается частями. Каждая часть файла отправляется как сообщение FILE_TRANSFER_CHUNK.

Сервер принимает части файла, записывает их во временный файл и параллельно считает SHA-256. После окончания передачи сервер сравнивает размер и контрольную сумму. Если проверка успешна, временный файл заменяет итоговый файл в хранилище.

Режим DMA

В режиме DMA клиент сначала отправляет служебное сообщение FILE_TRANSFER_START. В нем указаны имя файла, размер, контрольная сумма и режим передачи.

После подтверждения от сервера клиент передает содержимое файла через DefaultFileRegion.

В этом режиме содержимое файла не передается как JSON-сообщения. Сервер заранее знает размер файла из FILE_TRANSFER_START и завершает прием после получения нужного количества байт.

Проверка базовой синхронизации

Для первой проверки использовались три файла:

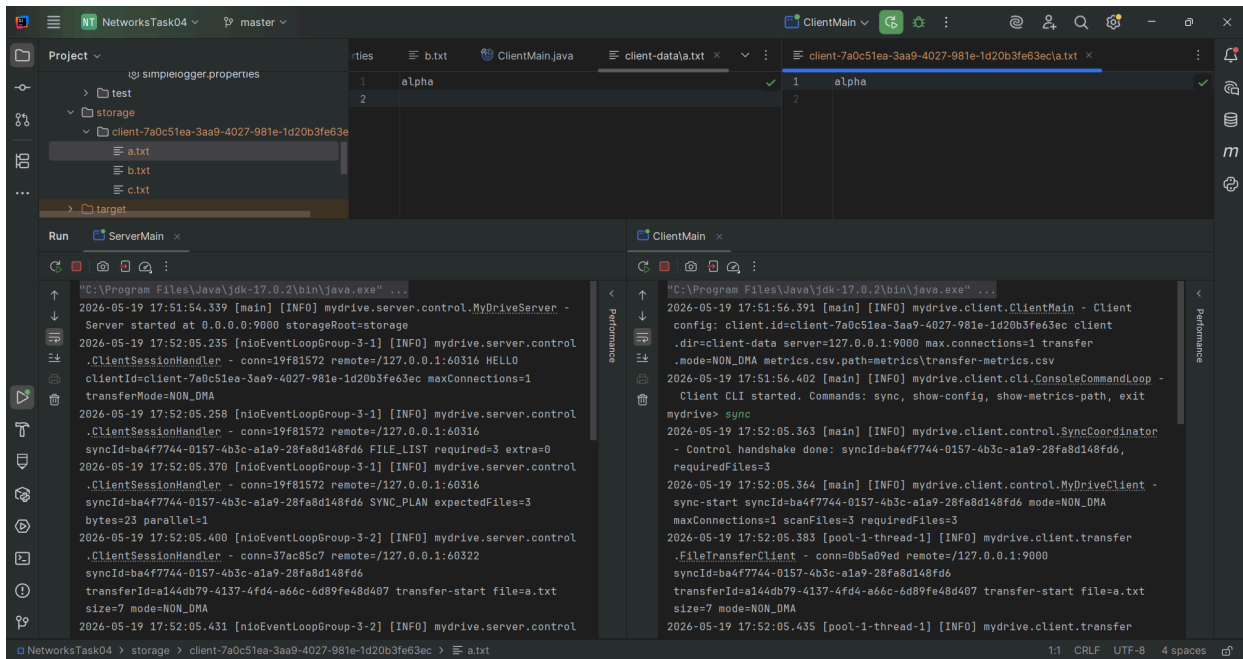
- a.txt;
- b.txt;
- c.txt.

Клиент был запущен с параметрами:

- max.connections=1;
- transfer.mode=NON_DMA.

После команды sync клиент отправил серверу список файлов. Сервер вернул, что нужно передать 3 файла. После этого клиент передал файлы на сервер.

На следующем скриншоте видны логи клиента и сервера. В логах сервера есть сообщения HELLO, FILE_LIST, SYNC_PLAN и начало передачи файла. В логах клиента видно, что синхронизация завершилась успешно.



TCP-обмен в WireShark

Для проверки TCP-обмена использовался WireShark. Так как клиент и сервер запускались на одной машине, захват выполнялся на loopback-интерфейсе.

Фильтр WireShark:

```
tcp.port == 9000
```

На следующем скриншоте видно, что клиент и сервер действительно обмениваются TCP-пакетами на порту 9000. В одном из пакетов виден

payload служебного сообщения.

The screenshot displays the Wireshark interface with a packet capture of a TCP connection. The packet list shows a sequence of packets, including a SYN, ACK, and data transfer. The packet details pane shows the structure of the TCP segment, including the header and payload. The packet bytes pane shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
329	7.910797700	127.0.0.1	127.0.0.1	TCP	145	60324 → 9000 [PSH, ACK] Seq=334 Ack=143 Win=65280 Len=89 TSval=64358214 TSecr=64358212
330	7.910859600	127.0.0.1	127.0.0.1	TCP	56	9000 → 60324 [ACK] Seq=143 Ack=423 Win=65024 Len=0 TSval=64358214 TSecr=64358214
331	7.911528800	127.0.0.1	127.0.0.1	TCP	314	60324 → 9000 [PSH, ACK] Seq=423 Ack=143 Win=65280 Len=258 TSval=64358215 TSecr=64358214
332	7.911586800	127.0.0.1	127.0.0.1	TCP	56	9000 → 60324 [ACK] Seq=143 Ack=681 Win=64768 Len=0 TSval=64358215 TSecr=64358215
333	7.914016700	127.0.0.1	127.0.0.1	TCP	312	9000 → 60324 [PSH, ACK] Seq=143 Ack=681 Win=64768 Len=256 TSval=64358217 TSecr=64358215

Frame 331: Packet, 314 bytes on wire (2512 bits), 314 bytes captured (2512 bits) on interface \Device\NPF_{...}_id 0

Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 60324, Dst Port: 9000, Seq: 423, Ack: 143, Len: 258

Source Port: 60324

Destination Port: 9000

[Stream Index: 6]

[Stream Packet Number: 10]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 258]

Sequence Number: 423 (relative sequence number)

Sequence Number (raw): 2675760518

[Next Sequence Number: 681 (relative sequence number)]

Acknowledgment Number: 143 (relative ack number)

Acknowledgment number (raw): 1211298283

1000 = Header Length: 32 bytes (8)

Flags: 0x018 (PSH, ACK)

Window: 255

[Calculated window size: 65280]

[Window size scaling factor: 256]

Checksum: 0xfc82 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

[Timestamps]

[SEQ/ACK analysis]

[Client Contiguous Streams: 1]

[Server Contiguous Streams: 1]

TCP payload (258 bytes)

Data (258 bytes)

Data [...]: 000000f0709003a888f1c1c6707b2273796e634964223a2262613466373734342d303135372d346233632d613161392d32386661386431...

Adapter for loopback traffic capture: <live capture in progress>

Пакеты: 2432 - Отображено: 68 (2.8%)

Профиль: Default

Проверка точной копии файлов

После синхронизации были сравнены файлы в клиентской директории и в серверном хранилище.

Для проверки использовались команды:

Get-ChildItem .\client-data -File | Select-Object Name,Length

Get-ChildItem .\storage -Recurse -File | Select-Object Name,Length

Get-FileHash .\client-data*.txt -Algorithm SHA256

Get-FileHash .\storage**.txt -Algorithm SHA256

На следующем скриншоте видно, что состав файлов и размеры совпадают. Также совпадает SHA-256 для файла a.txt.

```
PS C:\Users\ashat\OneDrive\Desktop\HSE\Networks\NetworksTask04> Get-ChildItem .\client-data -File | Select-Object Name,Length
Name Length
----
a.txt 7
b.txt 7
c.txt 9

PS C:\Users\ashat\OneDrive\Desktop\HSE\Networks\NetworksTask04> Get-ChildItem .\storage -Recurse -File | Select-Object Name,Length
Name Length
----
a.txt 7
b.txt 7
c.txt 9

PS C:\Users\ashat\OneDrive\Desktop\HSE\Networks\NetworksTask04>
PS C:\Users\ashat\OneDrive\Desktop\HSE\Networks\NetworksTask04> Get-FileHash .\client-data\*.txt -Algorithm SHA256
Algorithm Hash Path
-----
SHA256 4A31F8A329CDB8AB944056FC992EEB5AB5C0B4A78C3BC34E6D7C4A59FD6CF3B6 C:\Users\ashat\OneDrive\Desktop\HSE\Networks\NetworksTask04\client-data\*.txt

PS C:\Users\ashat\OneDrive\Desktop\HSE\Networks\NetworksTask04> Get-FileHash .\storage\*.txt -Algorithm SHA256
Algorithm Hash Path
-----
SHA256 4A31F8A329CDB8AB944056FC992EEB5AB5C0B4A78C3BC34E6D7C4A59FD6CF3B6 C:\Users\ashat\OneDrive\Desktop\HSE\Networks\NetworksTask04\storage\client-7a0c51ea-3aa9-4027-981e-1d20b3fe63ec\*.txt
```

Повторная синхронизация без изменений

После первой синхронизации команда `sync` была выполнена повторно без изменения файлов.

Сервер вернул:

- `requiredFiles=0;`
- `filesReceived=0;`
- `deletedExtraFiles=0.`

Это означает, что неизменные файлы повторно не передавались.

Проверка измененного файла

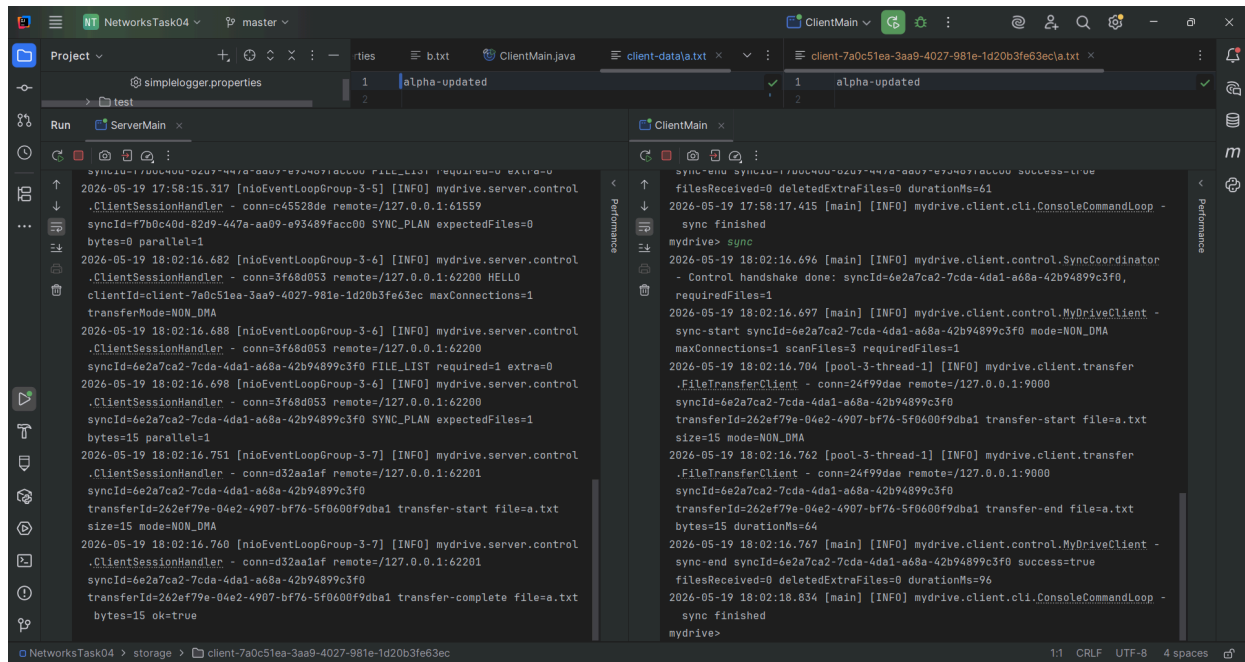
После этого файл `a.txt` был изменен на стороне клиента. Затем команда `sync` была выполнена снова.

Сервер определил, что изменился один файл:

- `requiredFiles=1.`

Клиент передал только файл `a.txt`.

На следующем скриншоте видно, что передавался только измененный файл. Синхронизация завершилась успешно.



```
2026-05-19 17:58:15.317 [nioEventLoopGroup-3-5] [INFO] mydrive.server.control
.ClientSessionHandler - conn=c45528de remote=/127.0.0.1:61559
syncId=f7b0c40d-82d9-447a-aa09-e93489facc00 SYNC_PLAN expectedFiles=0
bytes=0 parallel=1
2026-05-19 18:02:16.682 [nioEventLoopGroup-3-6] [INFO] mydrive.server.control
.ClientSessionHandler - conn=3f68d053 remote=/127.0.0.1:62200 HELLO
clientId=client-7a0c51ea-3aa9-4027-981e-1d20b3fe63ec maxConnections=1
transferMode=NON_DMA
2026-05-19 18:02:16.688 [nioEventLoopGroup-3-6] [INFO] mydrive.server.control
.ClientSessionHandler - conn=3f68d053 remote=/127.0.0.1:62200
syncId=6e2a7ca2-7cda-4da1-a68a-42b94899c3f0 FILE_LIST required=1 extra=0
2026-05-19 18:02:16.698 [nioEventLoopGroup-3-6] [INFO] mydrive.server.control
.ClientSessionHandler - conn=3f68d053 remote=/127.0.0.1:62200
syncId=6e2a7ca2-7cda-4da1-a68a-42b94899c3f0 SYNC_PLAN expectedFiles=1
bytes=15 parallel=1
2026-05-19 18:02:16.751 [nioEventLoopGroup-3-7] [INFO] mydrive.server.control
.ClientSessionHandler - conn=d32aa1af remote=/127.0.0.1:62201
syncId=6e2a7ca2-7cda-4da1-a68a-42b94899c3f0
transferId=262ef79e-04e2-4907-bf76-5f0600f9dba1 transfer-start file=a.txt
size=15 mode=NON_DMA
2026-05-19 18:02:16.760 [nioEventLoopGroup-3-7] [INFO] mydrive.server.control
.ClientSessionHandler - conn=d32aa1af remote=/127.0.0.1:62201
syncId=6e2a7ca2-7cda-4da1-a68a-42b94899c3f0
transferId=262ef79e-04e2-4907-bf76-5f0600f9dba1 transfer-complete file=a.txt
bytes=15 ok=true

syncId=f7b0c40d-82d9-447a-aa09-e93489facc00 success=true
filesReceived=0 deletedExtraFiles=0 durationMs=61
2026-05-19 17:58:17.415 [main] [INFO] mydrive.client.cli.ConsoleCommandLoop -
sync finished
mydrive> sync
2026-05-19 18:02:16.696 [main] [INFO] mydrive.client.control.SyncCoordinator
- Control handshake done: syncId=6e2a7ca2-7cda-4da1-a68a-42b94899c3f0,
requiredFiles=1
2026-05-19 18:02:16.697 [main] [INFO] mydrive.client.control.MyDriveClient -
sync-start syncId=6e2a7ca2-7cda-4da1-a68a-42b94899c3f0 mode=NON_DMA
maxConnections=1 scanFiles=3 requiredFiles=1
2026-05-19 18:02:16.704 [pool-3-thread-1] [INFO] mydrive.client.transfer
.FileTransferClient - conn=24f99dae remote=/127.0.0.1:9000
syncId=6e2a7ca2-7cda-4da1-a68a-42b94899c3f0
transferId=262ef79e-04e2-4907-bf76-5f0600f9dba1 transfer-start file=a.txt
size=15 mode=NON_DMA
2026-05-19 18:02:16.762 [pool-3-thread-1] [INFO] mydrive.client.transfer
.FileTransferClient - conn=24f99dae remote=/127.0.0.1:9000
syncId=6e2a7ca2-7cda-4da1-a68a-42b94899c3f0
transferId=262ef79e-04e2-4907-bf76-5f0600f9dba1 transfer-end file=a.txt
bytes=15 durationMs=64
2026-05-19 18:02:16.767 [main] [INFO] mydrive.client.control.MyDriveClient -
sync-end syncId=6e2a7ca2-7cda-4da1-a68a-42b94899c3f0 success=true
filesReceived=0 deletedExtraFiles=0 durationMs=96
2026-05-19 18:02:18.834 [main] [INFO] mydrive.client.cli.ConsoleCommandLoop -
sync finished
mydrive>
```

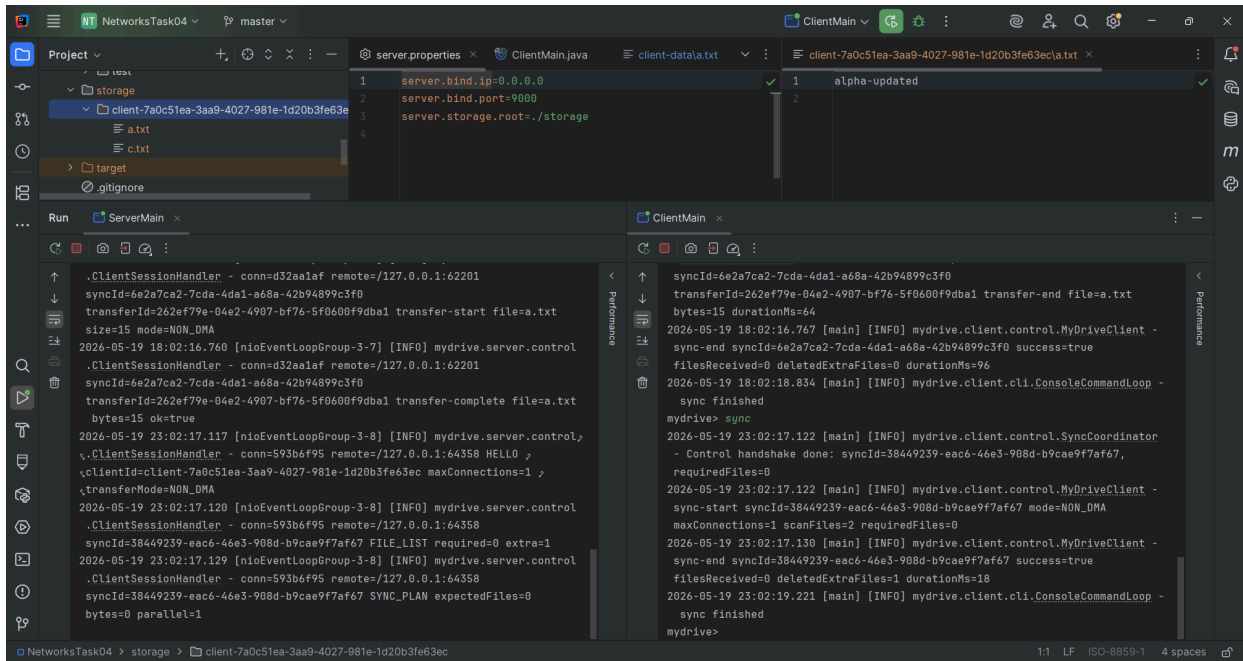
Удаление файла у клиента

Для проверки точной копии файл b.txt был удален из клиентской директории. После следующей синхронизации сервер удалил лишний файл из хранилища.

В логах видно:

- extra=1;
- deletedExtraFiles=1.

На следующем скриншоте видно, что после синхронизации файл b.txt отсутствует на сервере.

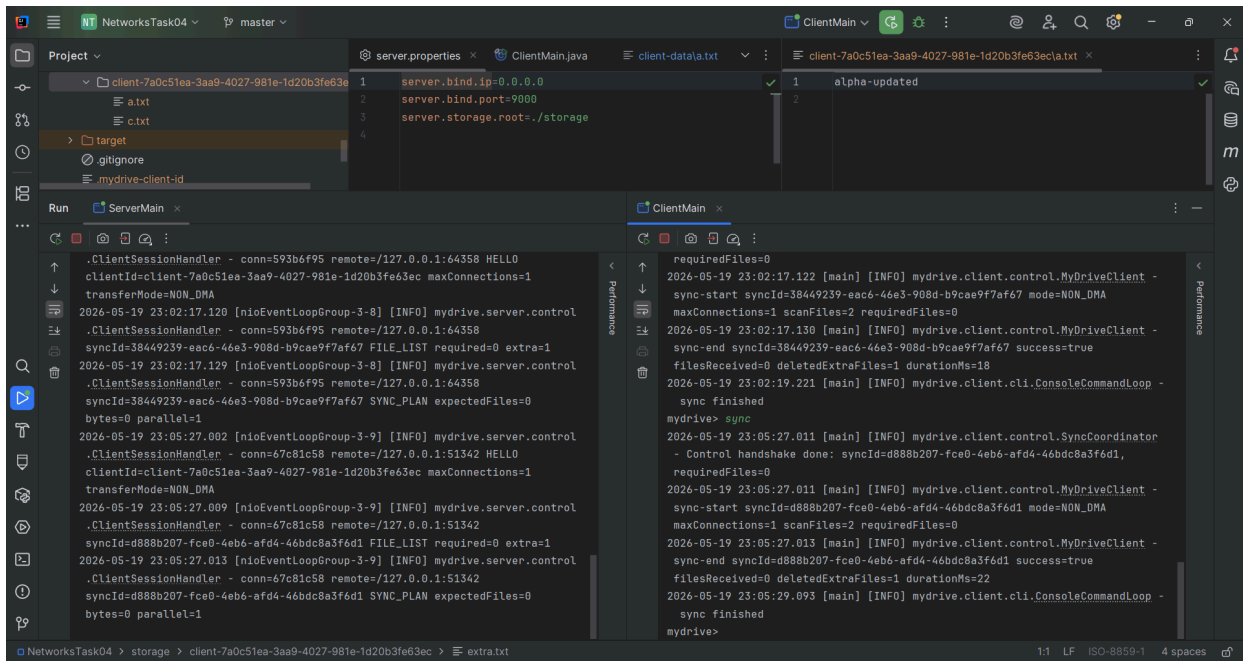


Удаление лишнего файла на сервере

Также был проверен случай, когда лишний файл был создан вручную на сервере. После следующей синхронизации сервер удалил этот файл.

В логах видно:

- extra=1;
- deletedExtraFiles=1.



Параллельная передача файлов

Для проверки параллельной передачи использовались 4 файла по 50 МБ. Клиент был запущен с параметром:

`max.connections=4`

В этом режиме клиент создает несколько TCP-соединений и отправляет несколько файлов одновременно.

На следующем скриншоте видны логи передачи. В логах есть разные conn, разные transferId и разные файлы. Это показывает, что файлы передавались

через разные соединения.

The screenshot shows an IDE with two tabs: `ClientMain` and `ServerMain`. The `ClientMain` tab displays the following code:

```
1 client.id=  
2 client.id.file= ./mydrive-client-id  
3 client.dir=./client-data  
4 server.ip=127.0.0.1  
5 server.port=9000  
6 max.connections=4  
7 transfer.mode=NON_DMA  
8 metrics.csv.path=./metrics/transfer-metrics.csv
```

The `ServerMain` tab shows logs for the `mydrive.server` application, including session handlers and file transfer events. The logs indicate successful transfers of files `p1.bin`, `p2.bin`, `p3.bin`, and `p4.bin` to the client.

В WireShark также видно несколько TCP-соединений к порту 9000. У каждого соединения свой клиентский порт.

The screenshot shows the Wireshark interface with the 'Conversations' pane selected. The table below lists the captured TCP connections to port 9000.

Адрес A	Порт A	Адрес B	Порт B	Пакеты	Байт	ИД потока	Всего пакетов	Отфильтровано в процентах	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Отк. время начала	Прс
127.0.0.1	49576	127.0.0.1	9000	17	3 кБ	1	17	100.00%	9	1 кБ	8	1 кБ	1.586995700	Прс
127.0.0.1	49577	127.0.0.1	9000	656	27 МБ	2	656	100.00%	422	27 МБ	234	14 кБ	1.603779100	
127.0.0.1	49578	127.0.0.1	9000	679	29 МБ	3	679	100.00%	457	29 МБ	222	13 кБ	1.603916100	
127.0.0.1	49579	127.0.0.1	9000	760	29 МБ	4	760	100.00%	462	29 МБ	298	17 кБ	1.603918700	

The bottom pane shows the details of the selected packet, including the calculated window size (65535), checksum (0xd221), and options (20 bytes, Maximum segment size, No-Operation (NOP), Window scale, SACK permitted, Timestamps).

Передача 10 файлов по 100 МБ

Для демонстрации работы с большими файлами были подготовлены 10 файлов. Размер каждого файла - 100 МБ.

Файлы:

- file01.bin;
- file02.bin;
- file03.bin;
- file04.bin;
- file05.bin;
- file06.bin;
- file07.bin;
- file08.bin;
- file09.bin;
- file10.bin.

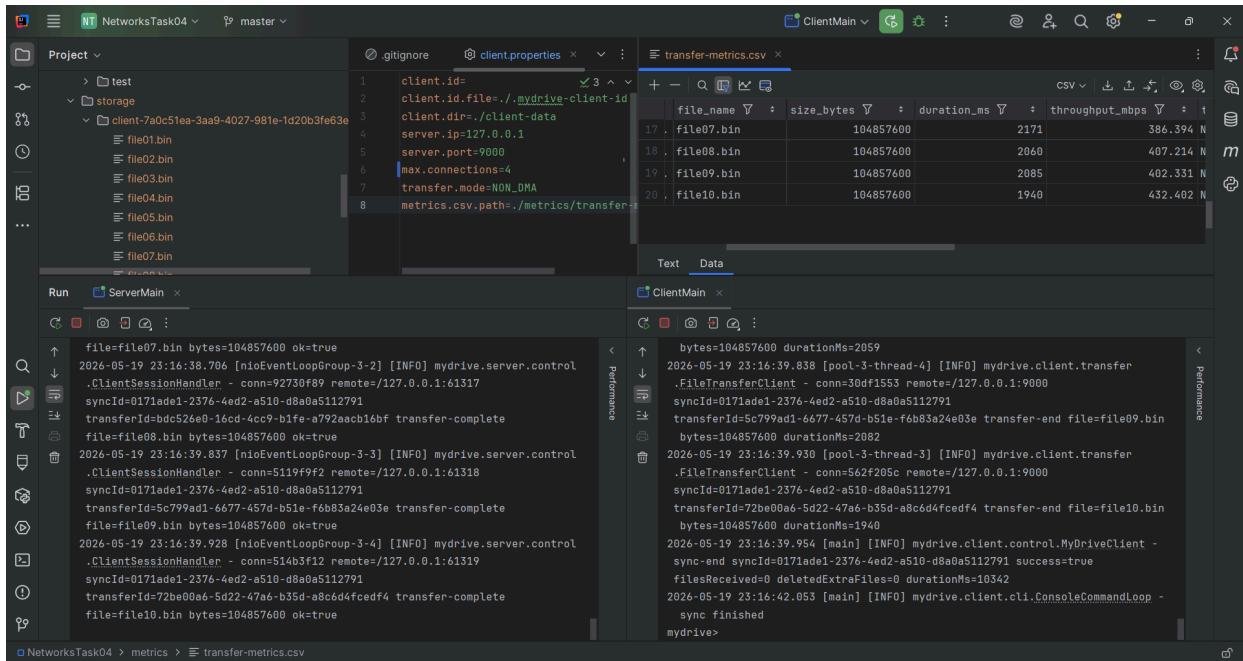
Размер каждого файла:

104857600 байт

Сначала проверка выполнялась в режиме NON_DMA с параметром:

max.connections=4

На следующем скриншоте видно, что файлы переданы на сервер. В таблице метрик есть время передачи и пропускная способность для файлов.



Передача файлов в режиме DMA

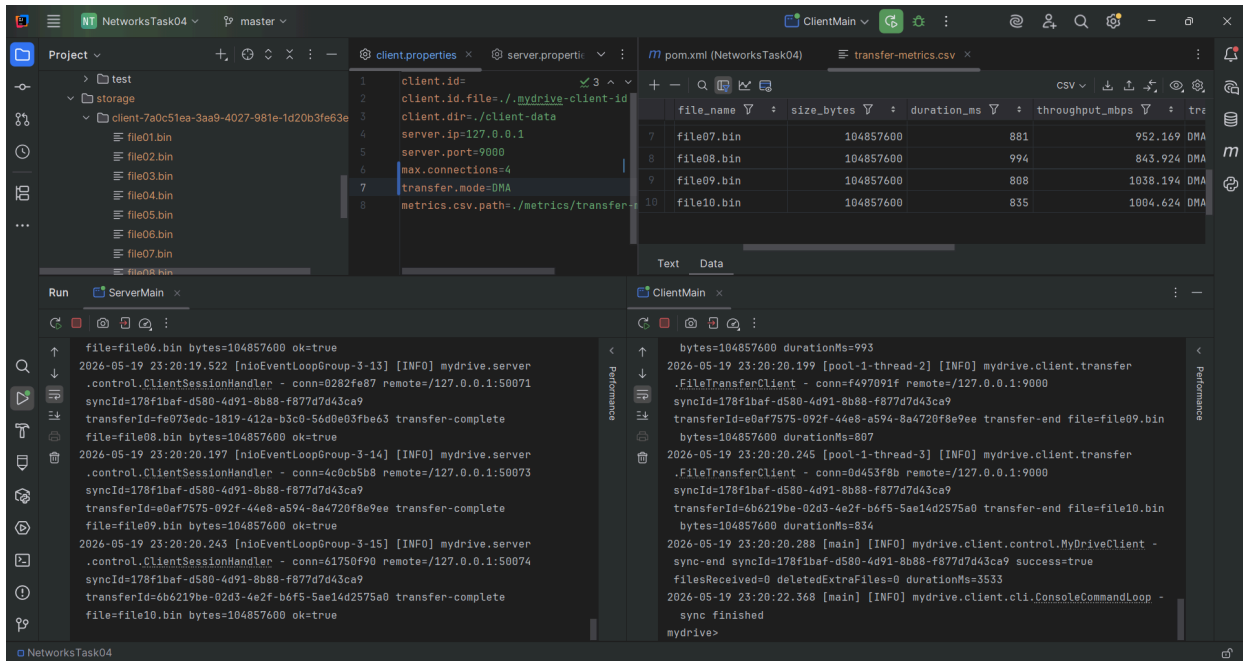
Затем те же 10 файлов были переданы в режиме DMA.

Параметры клиента:

max.connections=4

transfer.mode=DMA

В логах видно, что передача выполнялась в режиме DMA, сервер принял файлы, а синхронизация завершилась успешно.



Сравнение DMA и NON_DMA

Для сравнения времени передачи были использованы файлы размером:

- 50 МБ;
- 100 МБ;
- 250 МБ.

Для каждого файла передача выполнялась в двух режимах:

- NON_DMA;
- DMA.

На следующем скриншоте показаны CSV-файлы с результатами замеров. В таблицах есть размер файла, время передачи и пропускная способность.

dma_50_100_250.csv					non_dma_50_100_250.csv				
file_name	size_bytes	duration_ms	through		file_name	size_bytes	duration_ms	through	
file_100mb.bin	104857600	496			file_100mb.bin	104857600	1818		
file_250mb.bin	262144000	1145			file_250mb.bin	262144000	5170		
file_50mb.bin	52428800	266	3	3	file_50mb.bin	52428800	833		

Run ServerMain x ClientMain x

```
 syncId=f123eb7a-eec2-40f6-89d1-98936269880e transferId=ace70ad9-761f-44f7-9182-a15d75027315 transfer-start file=file_50mb.bin size=52428800 mode=DMA 2026-05-19 23:25:40.475 [nioEventLoopGroup-3-7] [INFO] mydrive.server.control .ClientSessionHandler - conn=88794672 remote=/127.0.0.1:65045 syncId=f123eb7a-eec2-40f6-89d1-98936269880e transferId=ace70ad9-761f-44f7-9182-a15d75027315 transfer-complete file=file_50mb.bin bytes=52428800 ok=true   
```

2026-05-19 23:25:40.487 [main] [INFO] mydrive.client.control.MyDriveClient - sync-end syncId=f123eb7a-eec2-40f6-89d1-98936269880e success=true
 filesReceived=0 deletedExtraFiles=0 durationMs=2447
 2026-05-19 23:25:42.578 [main] [INFO] mydrive.client.cli.ConsoleCommandLoop - sync finished
 mydrive>

По результатам замеров DMA оказался быстрее на локальном стенде. Это связано с тем, что в режиме DMA файл передается через DefaultFileRegion, а в режиме NON_DMA файл передается через служебные сообщения с чанками.

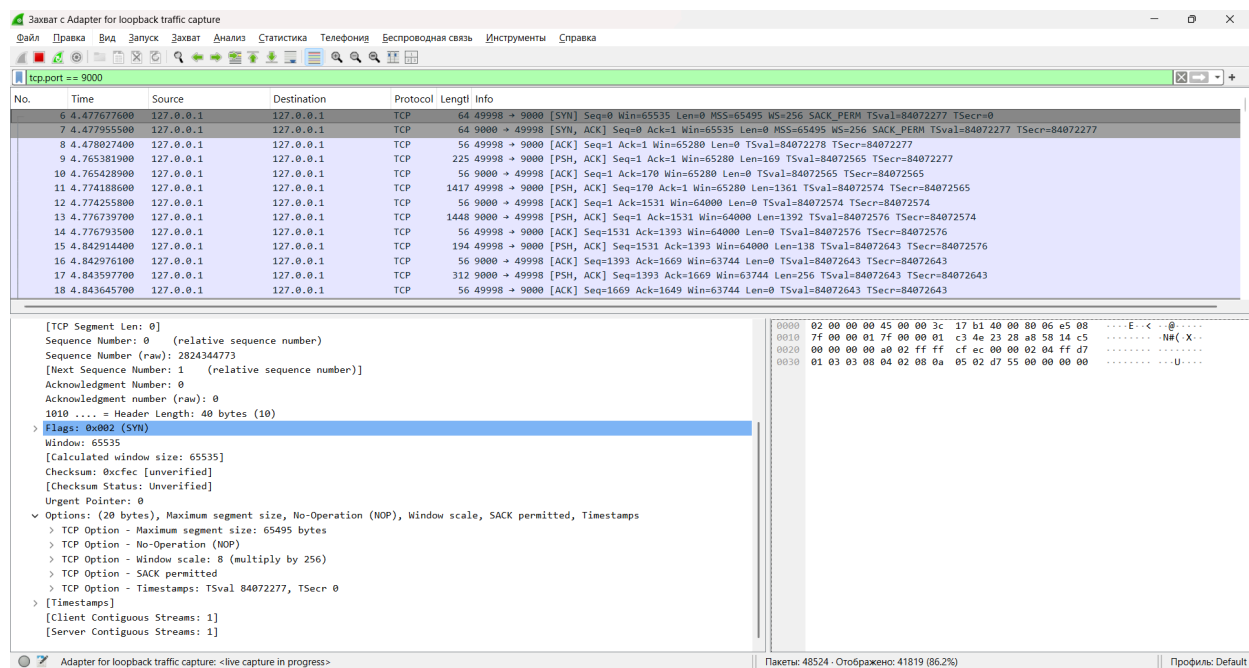
TCP handshake, MSS и Window

В WireShark был найден TCP handshake для соединения клиента с сервером.

В SYN-пакете наблюдались такие значения:

- MSS = 65495 байт;
- Window size value = 65535;
- Window scale = 256.

На следующем скриншоте виден TCP handshake и раскрытые TCP options.

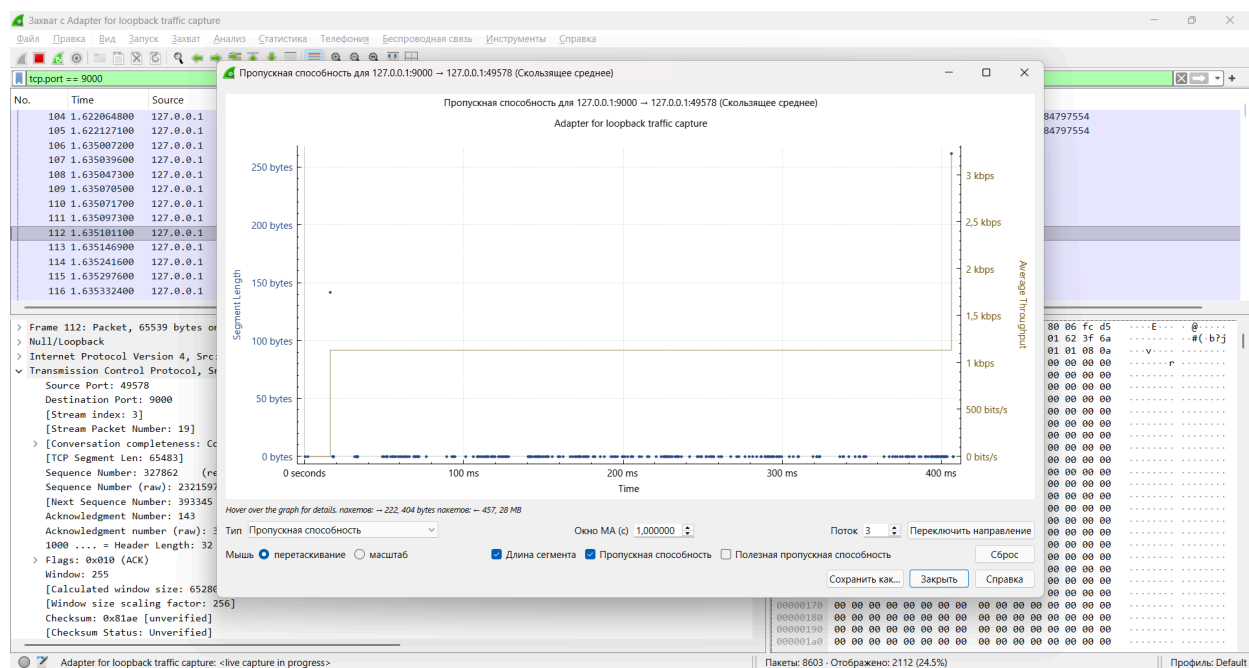


Наблюдения по TCP-потoku

Для одного из TCP-потокoв был построен график пропускной способности в WireShark.

Передача выполнялась через loorback-интерфейс. Из-за этого задержка была очень маленькой, а передача происходила быстро. Поэтому выраженный медленный старт визуально выделялся слабо.

На следующем скриншоте показан график для большого TCP-потoka.



Явных признаков congestion в локальном прогоне не наблюдалось. Retransmission и duplicate ACK в существенном объеме не были зафиксированы. Это ожидаемо для локального loopback-запуска, где нет реальной сетевой перегрузки.

Метрики передачи

Метрики записываются в файл:

metrics/transfer-metrics.csv

Для каждого файла записываются:

- sync_id;
- file_name;
- size_bytes;
- duration_ms;
- throughput_mbps;
- transfer_mode;
- max_connections;

- started_at_epoch_ms;
- finished_at_epoch_ms.

Эти данные использовались для сравнения режимов DMA и NON_DMA.

Итог

В работе реализовано клиент-серверное приложение MyDrive.

Клиент подключается к серверу по TCP, отправляет свой ID и список файлов. Сервер сравнивает файлы по имени, размеру и SHA-256. После этого клиент передает только недостающие или измененные файлы.

Сервер сохраняет файлы в отдельную директорию пользователя. После синхронизации серверная директория содержит точную копию файлов клиента.

Выполнено:

- параллельная передача файлов через несколько TCP-соединений;
- режим передачи NON_DMA;
- режим передачи DMA;
- сбор метрик передачи;
- проверка TCP-трафика в WireShark;
- анализ MSS, Window и TCP-потока.